Acknowledgments

First and foremost, I thank Professor Luca Saglietti for his invaluable guidance and constant availability. I also extend my gratitude to my mom, dad, Cristian, Marzia, and Grandma Marina for their unconditional support.

Special thanks to my colleagues Valentina, Luca, Michele, Giulia, the Edoardos and Andreas, Fabio, and Mattia for making our study sessions more enjoyable and stress-free. I am also grateful to my friends William, Mattia, Alessia, Sam, Tede, Luca, Sandro, Cippo, Antonio, Nicholas, Lori, Succe, Samu, Paolo, and Momo for their presence throughout these years and their GPUs availability. An honorable mention goes to BaoBing.

This work is dedicated to Grandpa Cesare.

Table of Contents

1 Introduction **3**

1.1 Settings 3

1.2 Example - Variational Autoencoders 4

1.2.1 Autoencoders 4

1.2.2 VAEs 6

1.3 Outline of the survey 7

2 Score based generative models **8**

2.1 Score matching 8

2.2 Practical issues 9

2.3 Training 10

2.4 Sampling 11

2.5 Motivation 12

3 Diffusion Probabilistic Models **13**

3.1 Definition 13

3.2 Forward process 13

3.3 Reverse process 14

3.4 Training 18

4 Dynamical Regimes of Diffusion Models **19**

4.1 Model definition 19

4.2 Main Results 20

4.3 Gaussian mixtures 22

4.3.1 Speciation time 23

4.3.2 Collapse time 25

5 Numerical Simulations **27**

5.1 Datasets 27

5.2 Model architecture and training 28

5.3 Methods 29

5.3.1 Speciation 30

5.3.2 Collapse 31

5.4 Results and considerations 31

5.4.1 Speciation 32

5.4.2 Collapse 33

6 Conclusions **35**

1 Introduction

Over the past few years, deep generative models have gained the ability to generate human-like natural language, high-quality realistic synthetic images, and highly diverse human speech and music. These models can be used in a variety of ways, such as generating images from text prompts (just think of recent software like Stable Diffusion and DALL-E) or learning useful high-level feature representations. Generative models could have wide-ranging impacts on graphic design, games, music production, and countless other fields.

In particular, the practical success achieved by generative diffusion models has already been broadly recognized, and it would be convenient to have an exact theoretical understanding of their behavior since they are going to be widely used in the future. We will begin this study by covering the basics of diffusion-based generative processes and some of the most important related models, then we will shift our focus to a recent paper [1] that attempted to explain their dynamics using statistical physics techniques.

1.1 Settings

To begin, we will outline the settings we will be working with.

Assume that we are working with images (we can generalize this argument to any type of data), let's first define our data as a set $\left\{\mathbf{x}_i \in \mathbb{R}^D\right\}_{i=1}^N$, where $\mathbf{x}_i$ corresponds to a single image/datapoint, N is the number of datapoints in the dataset, and D is the dimension in which each datapoint lives (that in our examples translates in the number of pixels of each image, multiplied by 3 if we are dealing with RGB images). Notice that although each image may have thousands of pixels, the set of images that represents meaningful objects is very small if we compare it to the set of all possible pixel combinations (also called the *ambient space*, indicated by $\mathbb{R}^D$).

We now suppose that the datapoints representing meaningful images all lie on a low dimensional manifold called *Data Manifold*. Notice that images that are similar or correlated between each other will be found in the same regions of the manifold, while different kinds of images (for example pictures of horses and pictures of cars) will belong to clusters distanced one another. We can go on and also suppose the existence of an unkown probability distribution $p_{data}(\mathbf{x})$ defined on the whole ambient space that measures how likely it is that a datapoint $\mathbf{x}$ belongs to the original dataset: it is going to be zero for datapoints that correspond to images that do not make sense and it will be positive for images that are on the data manifold (also, it is going to be low in regions of noisy images). Be aware that this probability function is not analytically derivable and it's very hard to track.
What we have just stated is the *Manifold Hypothesis*, and it is an assumption that is widely used in the context of data generation since empirically it has been shown to hold in many different contexts. All of the generative models that we are going to cover in this study have the task to approximate the data manifold and use it to find "new" images that were not present in the original dataset, or, equivalently, they are going to approximate p_{data} and sample from it. To give an idea on how generative models exploit these properties of data, we will now show an example.

1.2 Example - Variational Autoencoders

The first example of a generative model that we are going to explore is the Variational Autoencoder, first presented in [6]. In order to be able to describe it appropriately, we first have to learn about normal autoencoders.

1.2.1 Autoencoders

Autoencoders are a type of artificial neural network used to learn efficient coding of unlabeled data by capturing its most important features. Their structure is composed

of two different models: the encoder, that is responsible of the compression the input into a low dimensional representation, and a decoder that takes the compressed data and attempts to reconstruct the original input.

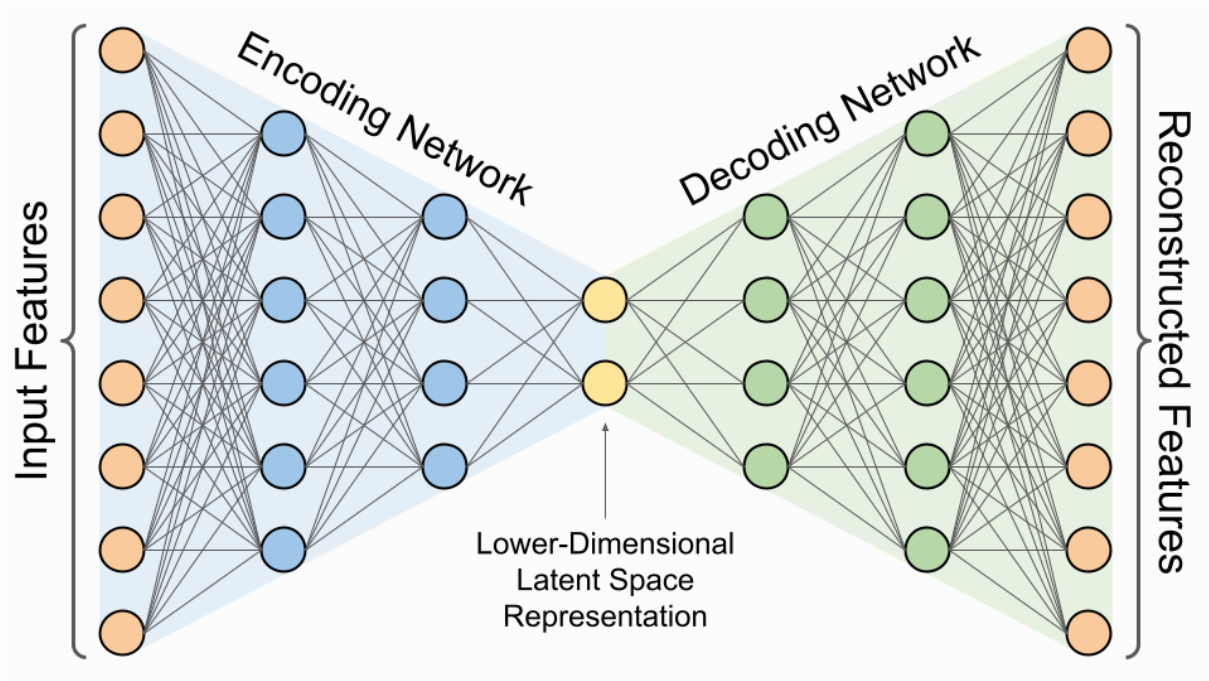


Figure 1: Image taken from https://www.assemblyai.com/blog/introduction-to-variational-autoencoders-using-keras/

The encoder is thus as a mapping from the inputs to a latent space that only stores the essential features necessary to reconstruct the original images, so we can see it as a transformation that preserves the data manifold (since they share the same features, datapoints on the manifold before the transformation end up on another manifold in the latent space). One could think that it may be a good idea to decide to sample from the latent manifold instead as it may be simpler to identify its distribution, but actually there are a few problems with this approach.

First of all it has been shown that the manifold learnt by the autoencoder is unstructured, meaning that points that are very close in the latent space may represent images that are actually very different. This means that we cannot really manipulate the latent representations in a way that produces coherent variations in the generated images. Secondly, the autoencoder is a deterministic algorithm, meaning that for each

coherent sample that we want to generate, we have to find a suitable latent datapoint (that is already a difficult task for the previous reason).

Luckily, all these problems are circumvented by Variational Autoencoders.

1.2.2 VAEs

Variational Autoencoders (VAE) [6] are a variant of normal autoencoders. Instead of training them to learn a code for some input **x**, we make them learn the parameters of a probability distribution defined on the latent space that is likely to generate **x**. The probability distribution we decide to use is, for variuos reasons, the multivariate Gaussian. The architecture is the following

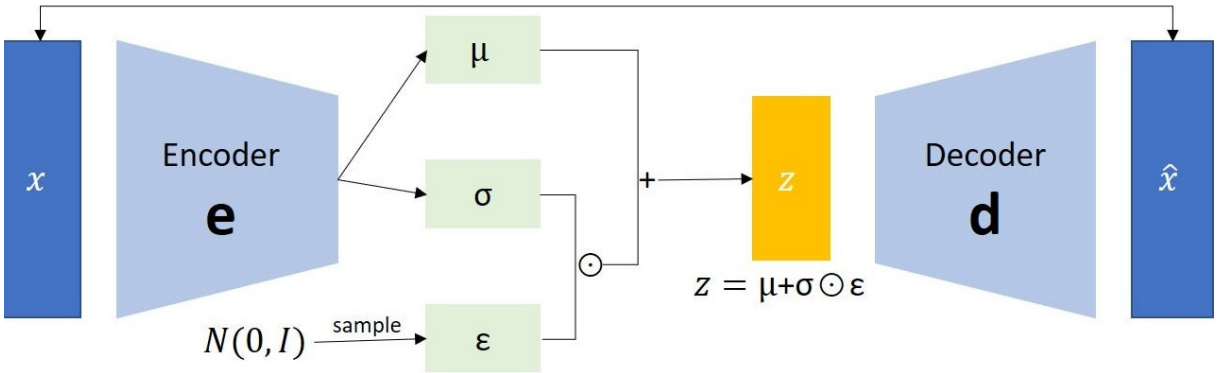


Figure 2: Image taken from https://medium.com/geekculture/variational-autoencoder-vae-9b8ce5475f68

What we want to do is to force the preservation of the structure of the manifold after the encoding transformation: mapping the image to a distribution is equivalent to mapping it to a whole latent region containing representations of very similar images. Also, sampling is made easier by the fact that the entire latent space is approximated by a standard normal distribution (the reasons can be found in the loss function used for training, that we are not going to discuss): meaning that sampling from the data distribution is equivalent to sampling from a standard Gaussian in the latent space. Since this is not a diffusion-based model, we decided to not go into its details. We include it in this survey so that we could show that there exist generative algorithms

that do not entirely rely on diffusion techniques.

1.3 Outline of the survey

The final objective of this survey is to be able to understand the importance of the contribution of the paper ”Dynamical Regimes of Diffusion Models” [1] to the research on diffusion models. In order to do so we will first have to go through a few of the most important historical examples of deep diffusion-based generative models, limiting our analysis to the ones that are directly correlated to the scope we just illustrated. Once this groundwork is laid, we will proceed with the paper analysis.
Before providing a formal theoretical definition of Diffusion Models we are going to first take a quick look at Score-Based generative models, that share some fundamental concepts with them.

2 Score based generative models

Given a dataset of independent identically distributed samples $\{\mathbf{x}_i \in \mathbb{R}^D\}_{i=1}^N$ and the "true" unknown distribution $p_{data}(\mathbf{x})$ let's suppose that, in order to sample from it, we want to use a gradient ascent-like approach: we start from any point $\mathbf{x}_0 \in \mathbb{R}^D$ and continuously follow the direction of the gradient $\nabla_{\mathbf{x}} p_{data}(\mathbf{x}_0)$ of the data distribution, hoping that it will make us move toward regions of the ambient space where $p_{data}(\mathbf{x})$ is higher (and $\mathbf{x}$ is more likely to come from the dataset). This is the idea used in Score Based Generative models, that were firstly introduced in [11].
Clearly, the hard part of this method is to find $\nabla_{\mathbf{x}} p_{data}(\mathbf{x})$ and define it on the whole $\mathbb{R}^D$. In order to do that, we are gonna implement *Score Matching* techniques [5].

2.1 Score matching

The idea is to approximate the *score* of the distribution $p(\mathbf{x}) = p_{data}(\mathbf{x})$, defined as $\nabla_{\mathbf{x}} \log p(\mathbf{x})$, using a neural network $\mathbf{s}_\theta : \mathbb{R}^D \to \mathbb{R}^D$ that we will call *Score Network* (notice that the logarithm function is strictly increasing, so maximizing $\log p(\mathbf{x})$ is equivalent to maximizing $p(\mathbf{x})$). We thus want to find the θ^* that minimizes

$$\frac{1}{2}\mathbb{E}_{p_{data}}\left[||\mathbf{s}_\theta(\mathbf{x}) - \nabla_{\mathbf{x}} \log p(\mathbf{x})||^2\right] = \mathbb{E}_p\left[tr(\nabla_{\mathbf{x}}\mathbf{s}_\theta(\mathbf{x})) + \frac{1}{2}||\mathbf{s}_\theta(\mathbf{x})||^2\right] \tag{1}$$

Where $\nabla_{\mathbf{x}}\mathbf{s}_\theta(\mathbf{x})$ is the Jacobian of $\mathbf{s}_\theta(\mathbf{x})$. The main problem is that its computation is not really scalable, as we would need to compute D partial derivatives (if we are dealing with images it is a high number) on a deep network. The method we implement in order to avoid this problem is *Denoising score matching* [14].
Denoising score matching is a variant of score matching that entirely avoids the term $tr(\nabla_{\mathbf{x}}\mathbf{s}_\theta(\mathbf{x}))$ firstly by adding noise to the data point x according to a gaussian kernel

$q_\sigma(\tilde{\mathbf{x}}|\mathbf{x})$, and then by by minimizing the following expression in θ:

$$\mathbb{E}_{\tilde{\mathbf{x}}\sim q_\sigma(\tilde{\mathbf{x}}|\mathbf{x})p_{\text{data}}(\mathbf{x})}\left[\|\mathbf{s}_\theta(\tilde{\mathbf{x}}) - \nabla_{\tilde{\mathbf{x}}}\log q_\sigma(\tilde{\mathbf{x}}|\mathbf{x})\|^2\right] \tag{2}$$

It can be shown that this minimization in θ is equivalent to the one in 1. Notice that $\tilde{\mathbf{x}}$ is our image $\mathbf{x}$ after the noising, and $\nabla_{\tilde{\mathbf{x}}}\log q_\sigma(\tilde{\mathbf{x}}|\mathbf{x})$ is the direction in which, starting from $\tilde{\mathbf{x}}$, we have to move to reach the original image $\mathbf{x}$. Intuitively $\tilde{\mathbf{x}}$ is moved outside of our data-manifold, and the network should give us the direction in which we have to move to get back to it. We will later see that this gradient is easily computable. (Actually, Denoising score matching isn't the only technique available: another method that may work for instance is Sliced score matching [13], which approximates $tr(\nabla_{\mathbf{x}}\mathbf{s}_\theta(\mathbf{x}))$ using random projections).

2.2 Practical issues

Before deciding how to define the dynamic, it is important to notice one problem that prevents our approach to work.

As we said before our data manifold constitutes only a very small part of the whole high dimensional space, and we expect the existence of very large regions $\mathcal{R} \subset \mathbb{R}^D$ such that $p_{data}(\mathcal{R}) \simeq 0$ (the support of the distribution is only the manifold) meaning that the gradient will vanish and, practically speaking, $\{\mathbf{x}_i\}_{i=1}^n \bigcap \mathcal{R} = \emptyset$. This means that in those regions we won't be able to estimate the score effectively, as shown in Figure 3.

To solve this problem, we are going to perturb data with Gaussian noise of different intensities: the reason is that, by adding noised images to our dataset, we are expanding our manifold to fill the low-density regions of the original ambient space (using large noise, score matching may get more training signal to improve score estimation even very far from the data manifold). If we apply different levels of noise, we

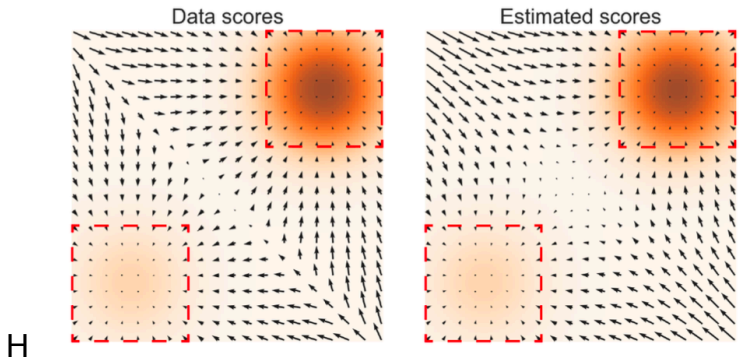


Figure 3: Image taken from [11]. On the left are plotted true Gaussian scores, on the right the estimated ones. The orange color highlights regions where the scores are similar. Notice that they are similar only when computed on points close to the high density regions of the Gaussian distributions.

may obtain a sequence of noised distributions that eventually converge to the true $p_{data}(\mathbf{x})$.

2.3 Training

Given the considerations we just stated, it may be a good idea to perform the training simultaneously with the perturbation of data. We first choose the noise levels $\{\sigma_i\}_{i=1}^{L}$ as a positive geometric sequence such that $\frac{\sigma_1}{\sigma_2} = ... = \frac{\sigma_{L-1}}{\sigma_L} > 1$. We want to choose σ_1 large enough to avoid the previously considered issues and σ_L small enough to have the minimum effect on data.

Following the Denoising score matching approach, we train the network to estimate the scores of all the perturbed distributions:

$$\mathbf{s}_\theta(\tilde{\mathbf{x}}, \sigma) \simeq \nabla_{\tilde{\mathbf{x}}} \log q_\sigma(\tilde{\mathbf{x}})$$

Where $q_\sigma(\tilde{\mathbf{x}}) = \int p_{data}(\mathbf{x}) q_\sigma(\tilde{\mathbf{x}}|\mathbf{x}) d\mathbf{x}$. Define $\mathbf{s}_\theta(\mathbf{x}, \sigma)$ as the *Noise conditional score network* (NCSN). Recall that

$$q_\sigma(\tilde{\mathbf{x}}|\mathbf{x}) \sim \mathcal{N}(\mathbf{x}, \sigma^2 I) \Rightarrow q_\sigma(\tilde{\mathbf{x}}|\mathbf{x}) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(\tilde{\mathbf{x}} - \mathbf{x})^2}{2\sigma^2}\right) \Rightarrow \nabla_{\tilde{\mathbf{x}}} \log q_\sigma(\tilde{\mathbf{x}}|\mathbf{x}) = -\frac{\tilde{\mathbf{x}} - \mathbf{x}}{\sigma^2}$$

We can already notice that, if we see $\tilde{\mathbf{x}}$ as the noised version of $\mathbf{x}$ after a passage of

the Gaussian kernel, we can say $\tilde{\mathbf{x}} = \mathbf{x} + \sigma\mathbf{z}$ where $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, I)$ and 2 becomes

$$\mathbb{E}_{\tilde{\mathbf{x}}\sim q_\sigma(\tilde{\mathbf{x}}|\mathbf{x})p_{\text{data}}(\mathbf{x})}\left[\|\mathbf{s}_\theta(\tilde{\mathbf{x}},\sigma) + \frac{\mathbf{x}+\sigma\mathbf{z}-\mathbf{x}}{\sigma^2}\|^2\right] = \mathbb{E}_{\tilde{\mathbf{x}}\sim q_\sigma(\tilde{\mathbf{x}}|\mathbf{x})p_{\text{data}}(\mathbf{x})}\left[\|\mathbf{s}_\theta(\tilde{\mathbf{x}},\sigma) + \frac{\mathbf{z}}{\sigma}\|^2\right] \tag{3}$$

meaning that the network is learning to predict the noise (the gradient is moving in the opposite direction of the noise) given $\tilde{\mathbf{x}}$ and the noise level σ.

We can now define the loss functions in the following way:

$$l(\theta;\sigma) := \frac{1}{2}\mathbb{E}_{p_{data}}\mathbb{E}_{\tilde{\mathbf{x}}\sim\mathcal{N}(\mathbf{x},\sigma^2 I)}\left[\|\mathbf{s}_\theta(\tilde{\mathbf{x}},\sigma) + \frac{\tilde{\mathbf{x}}-\mathbf{x}}{\sigma^2}\|^2\right]$$

$$\mathcal{L}(\theta;\{\sigma_i\}_{i=1}^L) := \frac{1}{L}\sum_{i=1}^{L}\lambda(\sigma_i)l(\theta;\sigma_i)$$

Where $l(\theta;\sigma)$ is the loss function for a specific noise level, and $\mathcal{L}(\theta;\{\sigma_i\}_{i=1}^L)$ is the unified objective. $\lambda(\sigma_i)$ is a coefficient greater than zero that makes sure that all the terms of the sum are of the same order: in fact it has been empirically observed that, after training, $||\mathbf{s}_\theta(\mathbf{x},\sigma)|| \propto \frac{1}{\sigma}$. We choose $\lambda(\sigma) = \sigma^2$ so that the terms in the sum do not depend on σ: in fact $||\mathbf{s}_\theta(\mathbf{x},\sigma)||$ becomes proportional to 1 and $\sigma\frac{\tilde{\mathbf{x}}-\mathbf{x}}{\sigma^2}$ becomes a standard normal distribution.

Now that the training objective is set, we can proceed to illustrate the sampling procedure.

2.4 Sampling

As sampling method we propose to use a Langevin Dynamic of the following type

$$\tilde{\mathbf{x}}_t = \tilde{\mathbf{x}}_{t-1} + \frac{\alpha}{2}\nabla_{\mathbf{x}}\log p(\tilde{\mathbf{x}}_{t-1}) + \sqrt{\alpha}\mathbf{z}_t$$

such that $\mathbf{z}_t \sim \mathcal{N}(\mathbf{0}, I)$ and $\alpha > 0$ is the step size. A nice property of this type of dynamics is that when $\epsilon \to 0$ and $T \to \infty$ (given that T is the number of iterations) the distribution of $\tilde{\mathbf{x}}_T$ equals $p(\mathbf{x})$, meaning that $\tilde{\mathbf{x}}_T$ becomes an exact sample of the true distribution [15] (notice that we have this guarantees only with the assumption of

Algorithm 1 Annealed Langevin dynamics.

Require: $\{\sigma_i\}_{i=1}^{L}, \epsilon, T$.

Initialize $\tilde{\mathbf{x}}_0$
for $i \leftarrow 1$ to L **do**
  $\alpha_i \leftarrow \epsilon \cdot \sigma_i^2 / \sigma_L^2$ ▷ α_i is the step size.
  for $t \leftarrow 1$ to T **do**
    Draw $\mathbf{z}_t \sim \mathcal{N}(0, I)$
    $\tilde{\mathbf{x}}_t \leftarrow \tilde{\mathbf{x}}_{t-1} + \frac{\alpha_i}{2} \mathbf{s}_{\boldsymbol{\theta}}(\tilde{\mathbf{x}}_{t-1}, \sigma_i) + \sqrt{\alpha_i}\, \mathbf{z}_t$
  end for
  $\tilde{\mathbf{x}}_0 \leftarrow \tilde{\mathbf{x}}_T$
end for
return $\tilde{\mathbf{x}}_T$

H

Figure 4: Algorithm from [11]

having access to the true score, but in practice we will replace it with the estimation given by our neural network).

Regarding the practical implementation, we use an annealing approach by gradually decreasing the step size. We initialize the samples from a prior noisy distribution, then run L consecutive Langevin dynamics such that each one corresponds to a certain noise level. Define the step-size $\alpha_i := \epsilon \frac{\sigma_i^2}{\sigma_L^2}$ and increase the index i every time we conclude a dynamic. The algorithm ends when we conclude the L-th dynamic. The idea is that each trajectory produces a sample that is closer to the original data manifold than the point at which the trajectory started, and it can be brought even closer by the following dynamic that is trained on a smaller noise level. In the end we expect to obtain a good quality sample from the original distribution.

2.5 Motivation

The reasons for the presentation of this model will be clear after the analysis of Diffusion Probabilistic models, that is the next topic that is going to be analyzed in this survey. We will notice resemblances between approaches that may seem very different in the beginning, but that in reality end up doing very similar things.

3 Diffusion Probabilistic Models

3.1 Definition

The essential idea behind diffusion models is to systematically and slowly destroy structure in a data distribution through an iterative forward diffusion process, so that we can learn, using a deep neural network, a reverse diffusion process that restores structure and information (we cannot derive it analytically, as the true distribution of data is unknown). When the network for removing noise is learned properly we can start off with completely random noise and let the model cook until we have a new image which could occur in our training data. Notice that, also in this context, we will exclusively work on image data. More formally, a diffusion model consists of three major components: the forward process, the reverse process and the sampling procedure.

This chapter is based on the work done in [4].

3.2 Forward process

Let $\mathbf{x}_0$ represent an image of our dataset, and label the data distribution as $q(\mathbf{x}_0)$. In the forward process we want to gradually convert the data distribution into complete noise, so we decide to repeatedly apply a Markov diffusion kernel $q(\mathbf{x}_t|\mathbf{x}_{t-1})$ for a diagonal Gaussian distribution. We also define $q(\mathbf{x}_{1:T}|\mathbf{x}_0)$ as the full *diffusion process* or *forward process*.

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) := \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t I), \qquad q(\mathbf{x}_{1:T}|\mathbf{x}_0) := \prod_{t=1}^{T} q(\mathbf{x}_t|\mathbf{x}_{t-1}) \tag{4}$$

Where $\beta_1, ..., \beta_T$ is a fixed variance schedule that normally is set to increase linearly from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$.

Notice that the forward process, thanks to that specific schedule, admits sampling $\mathbf{x}_t$

at any t in closed form. To see that, let's reparametrize $q(\mathbf{x}_t|\mathbf{x}_{t-1})$:

$$\mathbf{x}_t = \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\epsilon_{t-1}$$

Where $\epsilon_i \sim \mathcal{N}(0, \mathbf{I})\forall i = 1, ..., T-1$. Let's now define $\alpha_t := 1 - \beta_t$ and $\bar{\alpha}_t := \prod_{s=1}^{t} \alpha_s$, and substitute $\mathbf{x}_{t-1}$.

$$\mathbf{x}_t = \sqrt{\alpha_t}\mathbf{x}_{t-1} + \sqrt{1-\alpha_t}\epsilon_{t-1} = \sqrt{\alpha_t}(\sqrt{\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{1-\alpha_{t-1}}\epsilon_{t-2}) + \sqrt{\beta_t}\epsilon_{t-1} =$$
$$\sqrt{\alpha_t\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{\alpha_t(1-\alpha_{t-1})}\epsilon_{t-2} + \sqrt{1-\alpha_t}\epsilon_{t-1}$$

The last two terms represent a sum of Gaussians with different standard deviations, whose result is a Gaussian of the form $\mathcal{N}(\mathbf{0}, \alpha_t(1-\alpha_{t-1}) + 1 - \alpha_t)$.
Simplifying, we get

$$\mathbf{x}_t = \sqrt{\alpha_t\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{1-\alpha_t\alpha_{t-1}}\bar{\epsilon} = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$$

Where the last step has been done by induction. Notice that $\bar{\epsilon}, \epsilon \sim \mathcal{N}(\mathbf{0}, I)$. It is thus easy to sample $\mathbf{x}_t$ at any t (this is going to be useful later).

3.3 Reverse process

One of the reasons why we kept the step sizes of β very small, is that in this way the resulting true reverse process will have the same functional form of the forward process [3]. We can thus define the *backward process* $p_\theta(\mathbf{x}_{1:T}|\mathbf{x}_0)$ as a Gaussian, but with reversed time and parametrized mean and variance (we are looking for an approximation of the true backward process):

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \Sigma_\theta(\mathbf{x}_t, t)), \qquad p_\theta(\mathbf{x}_{1:T}) := p(\mathbf{x}_T)\prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) \tag{5}$$

We decide to fix the variance, since it has been shown that learning reverse process

variances leads to unstable training and poorer sample quality compared to training keeping fixed variances [1]. The variance will be set to β_t or $\tilde{\beta}_t = \beta_t \frac{1-\bar{\alpha}_t - 1}{1-\bar{\alpha}_t}$ (we will see later what $\tilde{\beta}_t$ is). We are thus left to find $\mu_\theta(\mathbf{x}_t, t)$, and our strategy to do so is to maximize the likelihood of $p_\theta(\mathbf{x}_0)$ over it (or, equivalently, minimize the negative log-likelihood).

First, notice that

$$p_\theta(\mathbf{x}_0) = \int d\mathbf{x}_{1:T} p_\theta(\mathbf{x}_{1:T})$$

is an integral over all the possible trajectories of $\mathbf{x}_t$ that from the "space of noisy data-points" end up in $\mathbf{x}_0$. That is clearly intractable and not computable.

We thus decide to minimize a lower bound instead: let's first define the KL divergence over two probability distributions f and g as

$$D_{KL}(f(x)||g(x)) := \int_{x \in \mathcal{X}} f(x) ln \left(\frac{f(x)}{g(x)} \right) dx = \mathbb{E}_f \left[ln \left(\frac{f(x)}{g(x)} \right) \right]$$

Notice that the KL divergence is never negative. We will use this fact at our advantage to find a lower bound on the negative log-likelihood that does not depend on $p_\theta(\mathbf{x}_0)$.

$$-ln(p_\theta(\mathbf{x}_0)) \leq -ln(p_\theta(\mathbf{x}_0)) + D_{KL}(q(\mathbf{x}_{1:T}|\mathbf{x}_0)||p_\theta(\mathbf{x}_{1:T}|\mathbf{x}_0))$$

The choice of the distributions will be clear very soon. We already notice that

$$p_\theta(\mathbf{x}_{1:T}|\mathbf{x}_0) = \frac{p_\theta(\mathbf{x}_{1:T}, \mathbf{x}_0)}{p_\theta(\mathbf{x}_0)} = \frac{p_\theta(\mathbf{x}_{0:T})}{p_\theta(\mathbf{x}_0)} \Rightarrow ln \left(\frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right) = ln \left(\frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0) p_\theta(\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} \right)$$

and

$$-ln(p_\theta(\mathbf{x}_0)) \leq -ln(p_\theta(\mathbf{x}_0)) + \mathbb{E}_q \left[ln \left(\frac{(q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T}))} \right) - ln(p_\theta(\mathbf{x}_0)) \right]$$

But $p_\theta(\mathbf{x}_0)$ doesn't depend on q, and thus we can move it outside of the expectation.

$$-ln(p_\theta(\mathbf{x}_0)) \leq -ln(p_\theta(\mathbf{x}_0)) + \mathbb{E}_q \left[ln \left(\frac{(q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T}))} \right) \right] - ln(p_\theta(\mathbf{x}_0)) = \mathbb{E}_q \left[ln \left(\frac{(q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T}))} \right) \right] = VLB$$

We just found the variational lower bound to minimize, and it is a KL divergence.

We can substitute (4) and (5) into the VLB to get

$$-ln(p_\theta(\mathbf{x}_T)) + \mathbb{E}_q\left[ln\left(\prod_{t=1}^{T}\frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}\right)\right] = -ln(p_\theta(\mathbf{x_T})) + \sum_{t=1}^{T}\mathbb{E}_q\left[ln\left(\frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}\right)\right] =$$
$$-ln(p_\theta(\mathbf{x_T})) + \mathbb{E}_q\left[\sum_{t=2}^{T}ln\left(\frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}\right) + ln\left(\frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1))}\right)\right] = VLB$$

Take a closer look at $q(\mathbf{x}_t|\mathbf{x}_{t-1})$ and notice that

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t)q(\mathbf{x}_t)}{q(\mathbf{x_{t-1}})} = \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0)q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x_{t-1}}|\mathbf{x}_0)}$$
$$\Rightarrow ln\left(\frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}\right) = ln\left(\frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}\right) + ln\left(\frac{q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x}_{t-1}|\mathbf{x}_0))}\right)$$

with that, we can split the sum in the expression for VLB. Let's look at the second term.

$$\sum_{t=2}^{T}ln\left(\frac{q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x}_{t-1}|\mathbf{x}_0))}\right) = ln\left(\prod_{t=2}^{T}\frac{q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x}_{t-1}|\mathbf{x}_0))}\right) = ln\left(\frac{q(\mathbf{x}_T|\mathbf{x}_0)}{q(\mathbf{x}_1|\mathbf{x}_0))}\right)$$

substituting and after a few changes, our final expression becomes

$$VLB = \mathbb{E}_q\left[ln\left(\frac{q(\mathbf{x}_T|\mathbf{x}_0)}{p_\theta(\mathbf{x}_T)}\right) + \sum_{t=2}^{T}ln\left(\frac{q(\mathbf{x}_t|\mathbf{x}_{t-1},\mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}\right) - ln(p_\theta(\mathbf{x}_0|\mathbf{x}_1))\right]$$

We can ignore both the first and the last term in the expectation as they don't take part into the optimization [4]. We are just left with a sum of KL divergences between $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ and the forward process posteriors conditioned on $\mathbf{x}_0$. This implies that, in order to minimize the variational lower bound, we have to parametrize $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ to make it as similar as possible to $q(\mathbf{x}_t|\mathbf{x}_{t-1},\mathbf{x}_0)$. What makes things easier is that $q(\mathbf{x}_t|\mathbf{x}_{t-1},\mathbf{x}_0)$ is actually analytically tractable (it is a Gaussian kernel). Let's prove this fundamental property before going on with the minimization.

$$q(\mathbf{x}_t|\mathbf{x}_{t-1},\mathbf{x}_0) = q(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0)\frac{q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|,\mathbf{x}_0)}$$
$$\propto exp\left(-\frac{1}{2}(\frac{(\mathbf{x}_t-\sqrt{\alpha_t}\mathbf{x}_{t-1})^2}{\beta_t} + \frac{(\mathbf{x}_{t-1}-\sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0)^2}{1-\bar{\alpha}_{t-1}}) - \frac{(\mathbf{x}_t-\sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0)^2}{1-\bar{\alpha}_t}\right)$$
$$= exp\left(-\frac{1}{2}((\frac{\alpha_t}{\beta_t}+\frac{1}{1-\bar{\alpha}_{t-1}})\mathbf{x}_{t-1}^2 - (\frac{2\sqrt{\alpha_t}}{\beta_t}\mathbf{x}_t + \frac{2\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}}\mathbf{x}_0)\mathbf{x}_{t-1} + \mathbf{C}(\mathbf{x}_t,\mathbf{x}_0))\right)$$

Just a few more steps are needed in order to make this exponential look like a gaussian: see the coefficient of $\mathbf{x}_{t-1}^2$ as the inverse of the variance and find the mean accordingly (change the coefficient of $\mathbf{x}_{t-1}$).

$$\tilde{\beta}_t = 1/(\frac{\alpha_t}{\beta_t} + \frac{1}{1-\bar{\alpha}_{t-1}}) = \beta_t \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}$$
$$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) = (\frac{\sqrt{\alpha_t}}{\beta_t}\mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}}\mathbf{x}_0)/(\frac{\alpha_t}{\beta_t} + \frac{1}{1-\bar{\alpha}_{t-1}}) = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}\mathbf{x}_0$$

But recall that $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\epsilon_t$, so $\mathbf{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}(\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\epsilon_t)$. We can thus remove the dependence on $\mathbf{x}_0$ by substituting its expression in $\tilde{\mu}_t$

$$\tilde{\mu}_t = \frac{1}{\sqrt{\bar{\alpha}_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_t\right)$$

For our purposes, let's suppose that this enough to show that $q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0) \sim \mathcal{N}(\tilde{\mu}_t, \tilde{\beta}_t I)$ is indeed a Gaussian distribution.

We can now go back to our VLB:

$$VLB = \sum_{t=2}^{T} D_{KL}(q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0)||p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))) =: \sum_{t=1}^{T-1} L_t$$

Both $q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0)$ and $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ are Gaussians, and thus we can compute the KL divergences in closed form. Up to a constant we have that

$$L_t = \mathbb{E}_{\mathbf{x}_0,\epsilon}\left[\frac{1}{2\sigma_t^2}||\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) - \mu_\theta(\mathbf{x}_t, t)||^2\right] \tag{6}$$

Where σ_t^2 is either β_t or $\tilde{\beta}_t$, depending on what we chose. We now want to train μ_θ to predict $\tilde{\mu}_t$, and by expanding the latter we get that we would like

$$\mu_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\bar{\alpha}_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_t\right)$$

to hold. Notice that $\mathbf{x}_t$ is already available, meaning that we can choose to approximate ϵ_t instead. We finally get $\mu_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\bar{\alpha}_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(\mathbf{x}_t, t)\right)$. By finding the mean of the backward process we finally got the last piece of our backward dynamic:

sampling $\mathbf{x}_{t-1} \sim p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ is thus equivalent to compute

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\bar{\alpha}_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(\mathbf{x}_t, t)\right) + \sigma_t \mathbf{z} \tag{7}$$

where $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, I)$ and $\epsilon_\theta(\mathbf{x}_t, t)$ has to be learned.

3.4 Training

In order to find a proper loss function, let's continue the minimization from above. Substituting both $\tilde{\mu}_t$ and μ_θ into 6 and after some simplification, we get that the objective becomes

$$\mathbb{E}_{\mathbf{x}_0,\epsilon}\left[\frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}||\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t)||^2\right] \tag{8}$$

Parametrizing the backward process then boils down to predicting ϵ given $\mathbf{x}_0$ and t, so we can just train our model by simply noising images of the dataset with intensities corresponding to a random t and trying to predict the noise.

Now take a moment and recall that we already saw a very similar result in equation 3, where the term in the expectation was $\|\mathbf{s}_\theta(\tilde{\mathbf{x}}, \sigma) + \frac{\mathbf{z}}{\sigma}\|^2$. It turns out that this minimization requires the prediction of the noise, exactly like the result we found in score matching: with respect to this notation we can see the old result as $\mathbf{s}_\theta \simeq -\frac{\epsilon_\theta(\mathbf{x}_t,t)}{\sqrt{1-\bar{\alpha}_t}}$. This means that the minimization in DDPMs resembles denoising score matching over multiple noise scales indexed by t (as if we are still learning the gradient). More formally, optimizing an objective that resembles denoise score matching is the same operation as using variational inference to fit the marginal of a sampling chain resembling a Langevin dynamics. Now that we managed to formalize Diffusion Models and to show some of their essential properties, like the equivalence with score matching, we are finally ready to perform the analysis of the paper [1].

Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$
6: **until** converged

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \ldots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

Figure 5: Algorithms from [4]

4 Dynamical Regimes of Diffusion Models

We will now develop a theoretical approach able to describe the dynamics of the types of diffusion models we just described. Our objective it to show that the backward generative dynamic, that starts from pure noise, first encounters a ”speciation” transition in which the category of the newly generated data is decided, and then a ”collapse” transition in which the trajectory of the dynamic becomes attracted to one of the original data points converging to it.

The full derivations of the results can be found in [1], and are achieved by means of statistical physics.

4.1 Model definition

We first assume that there exist two different classes of data, and each datapoint can only belong to one of them (it is possible to generalize the analysis to cases with multiple classes). Denote the datapoints as $\mathbf{a} \in \mathbb{R}^d$, assume the existence of a true distribution $P_0(\mathbf{a})$ for the datapoints, and define the empirical distribution of data as

$$P_0^e(\mathbf{a}) := \sum_{\mu=1}^{n} \frac{\delta(\mathbf{a} - \mathbf{a}_\mu)}{n}.$$

Also, denote Λ the highest eigenvalue of the covariance matrix of the data, as it will be useful later.

We decide to model our forward diffusion process as d independent Ornstein-Uhlenbeck

Langevin equations in the following way:

$$d\mathbf{x}(t) = -\mathbf{x}(t) + d\mathbf{B}(t) \tag{9}$$

Where $d\mathbf{B}(t)$ is the square root of 2 times the Brownian motion. Recall that this process satisfies the Fokker-Planck equation, whose solution for $P_t^e(\mathbf{x}(t))$ is

$$P_t^e(\mathbf{x}(t)) = \int d\mathbf{a} P_0^e(\mathbf{a}) \frac{1}{\sqrt{2\pi\Delta_t}^d} exp\left(-\frac{(\mathbf{x} - \mathbf{a}e^{-t})^2}{2\Delta_t}\right) \tag{10}$$

where $\Delta_t = 1 - e^{-2t}$. Notice that as t goes to infinity we have $P_t^e(\mathbf{x}(t)) \sim \mathcal{N}(\mathbf{0}, I)$, that is exactly the result of the forward process on images of the dataset.
To conclude our model description, we define the backward dynamic in a way to resemble the sampling algorithms of Score-based generative models (SBGM) and DDPMs:

$$-dy_i(t) = y_i dt + 2\mathcal{F}_i(y,t)dt + d\xi_i(t) \tag{11}$$

Where **y** is the starting point, $\mathcal{F}$ is the exact score function (that corresponds to the neural the network predictions) and $\xi(t)$ is the noise component (corresponding to the Langevin dynamic of Score Based Generative Models) distributed as in the forward process.

4.2 Main Results

This characterization of the model is enough to describe the backward dynamic as the sequence of three different dynamical regimes:

- the first one is basically pure Brownian motion that starts from a noisy distribution. In this regime it is very hard to distinguish any kind of structure of the image.

- In the second one the backward trajectory decides the main class of the data from which it is going to sample, and high level feature start to be visible.
- In the third regime, trajectories start to be attracted by the data point on which they will collapse at t = 0

Check Figure 6 for a nice visual representation.

We define *Speciation time* t_s the exact time of the transition between regime I and II, and *Collapse time* t_C the time of the transition between regimes II and III. It can be shown that $t_s = \frac{1}{2}\log\Lambda$ and that t_C satisfies $s(t_C) = s^{sep}(t_C)$, where $s(t)$ is the Shannon entropy per variable of the true distribution at time t, and $s^{sep}(t_C)$ will be defined in the next paragraphs. Let's take a closer look at the formulas.

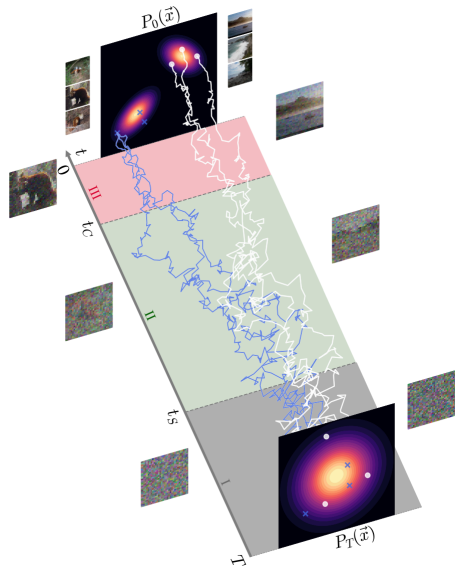


Figure 6: Image taken from [1]. Note that in the green region the trajectories begin to move apart.

See Λ as a measure of how much different the classes of data points in our dataset are. If t_s increases with it it means that if the classes are very different then the differentiation will happen very early in the dynamic, since the emerging of just a very few features would be enough to determine the class the we will end up choosing.

The discussion of the equation for t_C requires more work. First, let's notice that for small times, under the assumption of having the exact score, the support of the empirical distribution $P_t^e(\mathbf{x})$ is actually composed of "lumps" around the points of the training set (the forward dynamic has just started): this allows us to model it as some sort of Gaussian mixture with well separated centers, and we will define its entropy as

$s^{sep}(t_C)$. We know the closed form of the entropy of a Gaussian mixture of n clusters:

$$s^{sep}(t) = \frac{\log n}{d} + \frac{1}{2} + \frac{1}{2}\log(2\pi\Delta_t). \tag{12}$$

It thus makes sense to assume that the collapse transition will occur when the entropy of P_t will be equivalent to this one.

One important thing to notice is that the distributions of P_t and P_t^e are different for small t (P_t^e is distributed as lumps, P_t should look more continuous), but are actually very similar for $t >> 1$ as they both resemble n-dimensional Gaussians (whose entropy is $\frac{d}{2} + \frac{d}{2}\log(2\pi)$). This means that for $t >> 1$, the difference between $s^e(t)$ (defined on the empirical distribution) and $s^{sep}(t)$ equals $\frac{\log n}{d} =: \alpha$.

This new term actually tells us a lot about the collapse transition: if for instance α is very small then the collapse will occur very early in the dynamic, but by increasing it we can reduce t_C and shrink the length of the third regime. This is a really important fact to know when training a generative model: if we want our model to avoid memorization of the training dataset during sampling, then what we have to do is to have a number of samples n that is exponential in the number of dimensions d to increase α as much as possible.

We now use what we have just learned to analyze the case in which the data classes are modeled by a Gaussian mixture of two well separated cluster with means $\pm\vec{m}$ and variance σ^2. Later we will apply the same methods on general distributions and real datasets.

4.3 Gaussian mixtures

As anticipated, let's see $P_0(\vec{x})$ as the superposition of two Gaussian clusters of equal weight. Having access to the true distribution allows us to compute analytically both t_s and t_C. We will not cover the full derivations as it would take too much time and

space, but we will provide the main concepts.

4.3.1 Speciation time

Our strategy to compute the speciation time is the following:
first we sample a point $\mathbf{x}_f$ from a standard normal and then start a backward process until a certain time t at point $\mathbf{y}(t)$. We now perform a cloning and generate two distinct trajectories $\mathbf{x}_1(t)$ and $\mathbf{x}_2(t)$ starting from $\mathbf{y}$, and see what is the probability $\phi(t)$ that they will end up in the same class. We want to find this probability in function of t.
In order to compute it analytically, let's first define some useful quantities: $P(\mathbf{x}_1, 0|\mathbf{y}, t)$ as the probability of ending up in point $\mathbf{x}_1$ given that we started at $\mathbf{y}$ at time t, and

$$Q(\mathbf{x}_1, \mathbf{x}_2, \mathbf{y}, t) := P(\mathbf{x}_1, 0|\mathbf{y}, t)P(\mathbf{x}_2, 0|\mathbf{y}, t)P(\mathbf{y}, t) \tag{13}$$

as the probability of having two clones starting at $\mathbf{y}$ and ending up respectively in $\mathbf{x}_1$ and $\mathbf{x}_1$. In this way, we can find $\phi(t)$ by integrating $Q(\mathbf{x}_1, \mathbf{x}_2, \mathbf{y}, t)$ in $\mathbf{x}_1$, $\mathbf{x}_2$ and $\mathbf{y}$ assuming that $\mathbf{x}_1$ and $\mathbf{x}_2$ are of the same class, so within the constraint $(\mathbf{x}_1 \cdot \mathbf{m})(\mathbf{x}_2 \cdot \mathbf{m}) > 0$. Given that $P_t(\mathbf{x})$ is the convolution of the original distribution P_0 and the diffusion kernel $e^{\left(-\frac{(\mathbf{x}-\mathbf{a}e^{-t})^2}{2}\right)}$, we get

$$P_t(\mathbf{x}) = \frac{1}{(2\sqrt{2\pi\Gamma_t})^d}\left[e^{-(\mathbf{x}-\mathbf{m}e^{-t})^2/(2\Gamma_t)} + e^{(\mathbf{x}+\mathbf{m}e^{-t})^2/(2\Gamma_t)}\right]. \tag{14}$$

Where $\Gamma_t := \sigma^2 e^{-2t} + \Delta_t$. By putting everything together we notice that $\phi(t)$ can actually be reduced to a one dimensional integral

$$\phi(t) = \frac{1}{2}\int_{-\infty}^{+\infty} \frac{G(y, me^{-t}, \Gamma_t)^2 + G(y, -me^{-t}, \Gamma_t)^2}{G(y, me^{-t}, \Gamma_t) + G(y, -me^{-t}, \Gamma_t)}\, dy.$$

Where $G(y, u, v)$ is a Gaussian in the variable y with mean u and variance v. By looking at the behaviour of this function at zero and infinity, we find that $\phi(0) = 1$ and $\phi(\infty) = 1/2$, meaning that it is impossible to precisely predict the class in which we are going to end up if we start very far in time. From Figure 7 we can already deduce that $\phi(t)$ will behave like a step function around t_s in the limit of high dimensions.

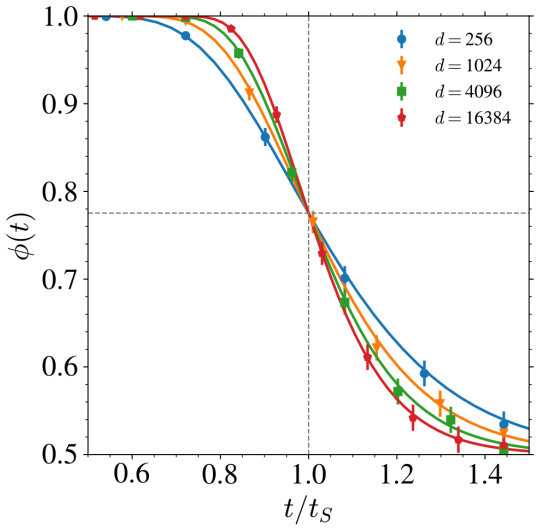


Figure 7: Image taken from [1].

We will now try to describe the mechanism of symmetry breaking that lies behind this phase transition. The first thing that we are going to do is to find the closed form of the backward dynamic, but first we need the score of $P_t(\mathbf{x})$ defined in equation 14.

$$\mathcal{F}_i(\mathbf{x}, \mathbf{t}) = -\frac{x_i}{\Gamma_t} + m_i \frac{e^{-t}}{\Gamma_t} \tanh\left(\mathbf{x} \cdot \mathbf{m} \frac{e^{-t}}{\Gamma_t}\right) \tag{15}$$

We can now substitute it in equation 11(the characterization of the backward dynamic that we introduced in this chapter) to get

$$-dx_i = x_i + 2\left(-\frac{x_i}{\Gamma_t} + m_i \frac{e^{-t}}{\Gamma_t} \tanh\left(q(t) \frac{e^{-(t-t_s)}}{\Gamma_t}\right)\right) dt + d\eta_i(t)$$

Where $q(t) = \mathbf{m} \cdot \mathbf{x}(t)/\sqrt{d}$ and t_s is approximated by $\log(d)/2$ (this approximation works fine for mulitivariate Gaussians in high enough dimensions). As usual, $d\eta_i(t)$ has the same distribution of the noise in the forward process. We now proceed to obtain the equation for $q(t)$.

$$-dq = -\left(q - 2\frac{|\mathbf{m}|^2}{d} e^{-t}\sqrt{d} \tanh(q e^{-t}\sqrt{d})\right) dt + d\eta_i(t).$$

If we look at the coefficient of dt as the gradient of a function, we can see this equation as a Langevin dynamic.

$$-dq = -\frac{\partial V(q,t)}{\partial q}dt + d\eta(t) \Rightarrow V(q,t) = \frac{1}{2}q^2 - 2\frac{|\mathbf{m}|^2}{d}\log\cosh(qe^{-t}\sqrt{d}).$$

If we look at big times $t >> t_s$ the second term disappears, and we are left with a quadratic potential. If instead we look at small times $t << t_S$ the potential develops a double well structure with a very large barrier, and the gradient makes sure that we don't escape the well we find ourselves in (in this case the speciation had already happened).

4.3.2 Collapse time

Assume we start our forward dynamic at $\mathbf{a}_1$, and describe it as $\mathbf{x} = \mathbf{a}_1 e^{-t} + \mathbf{z}\sqrt{\Delta_t}$ where $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, I)$. We now want to know if the score obtained from $P_t^e(\mathbf{x})$ makes the trajectory converge to $\mathbf{a}_1$ or if it allows for generalization. Let's first express equation 10 as:

$$P_t^e(\mathbf{x}) = \frac{Z_1 + Z_{2\ldots n}}{\sqrt{2\pi\Delta_t}^d}$$

Where

$$Z_1 = e^{-\frac{1}{2}(\mathbf{x}-\mathbf{a}_1 e^{-t})^2/\Delta_t}, Z_{2\ldots n} = \sum_{\mu=2}^{n} e^{-\frac{1}{2}(\mathbf{x}-\mathbf{a}_\mu e^{-t})^2/\Delta_t}$$

Let's now take the limit in both n and d, keeping $\alpha = \frac{\log n}{d}$ fixed: we have that $Z_1 \simeq e^{-d/2}$, while $Z_{2\ldots n}$ turns out to be equal to the partition function of a random energy model of glass transition. The result is that (again, all the calculations related to statistical physics are out of the scope of this study, and can be found in [1]) $Z_{2\ldots n}$ concentrates on an increasing function of time called $\psi_+(t)$. Analytically can be shown that there exist a value t_C (equivalent to the one defined before) such that for $t < t_C$ we have that $P_e^t(\mathbf{x})$ is dominated by the Z_1 term, meaning that the dynamic will converge almost surely on point $\mathbf{a}_1$ (this corresponds to memorization), but if $t > t_C$ the

opposite happens and we are in the generalization regime: meaning that we have the equivalence between the true and empirical distributions in every point **x**.

In this particular case of Gaussian mixtures, if we apply the collapse time equation, we get

$$t_C = \frac{1}{2}\log(1 + \frac{\sigma^2}{n^{2/d} - 1})$$

Also from here, we can see that in order to avoid the collapse we need to have n exponential in d to reduce the collapse time.

We finally completely characterized the behaviour of Gaussian mixtures, meaning that we are now ready to use this techniques on real data.

5 Numerical Simulations

Let's now move to the context in which P_0 is too complex to be analyzed exactly, but we can dispose of a dataset that is directly drawn from it.

The goal of this section is to numerically extract both collapse and speciation times and to compare them to the theoretical predictions, but before doing so we take a moment to describe the datasets and the model architecture used (all the code is available on [2]).

5.1 Datasets

We have to choose our datasets according to the fact that we are looking for data divided in two fairly different classes: our choices are going to be MNIST [8] (from which we take the classes of numbers 0 and 7), FashionMNIST [16] (pullovers and sneakers) and CIPHAR10 [7] (horses and cars).

Figure 8: From left to right: MNIST, FashionMNIST, CIPHAR10

We also chose the datasets so that we could see the different predicted behaviours of our model by changing number of samples n and number of dimensions d. These properties are actually shared by the MNIST datasets, but we keep them both to see if changes in the properties of the datasets without changing the dimensions will bring to any change in the behaviours of $\phi(t)$ (this was not explored in the original paper). Note that we limited the data processing at subtracting the mean in each channel.

5.2 Model architecture and training

As denoiser we choose a U-NET [10] that we are going to train to optimize the objective in equation 8. The architecture is similar to the ones used in previous applications of Denoising Diffusion Probabilistic Models [4]: the main difference with respect to basic U-NETs is the presence of positional embeddings that the network uses to keep track of time. Let's visualize it.

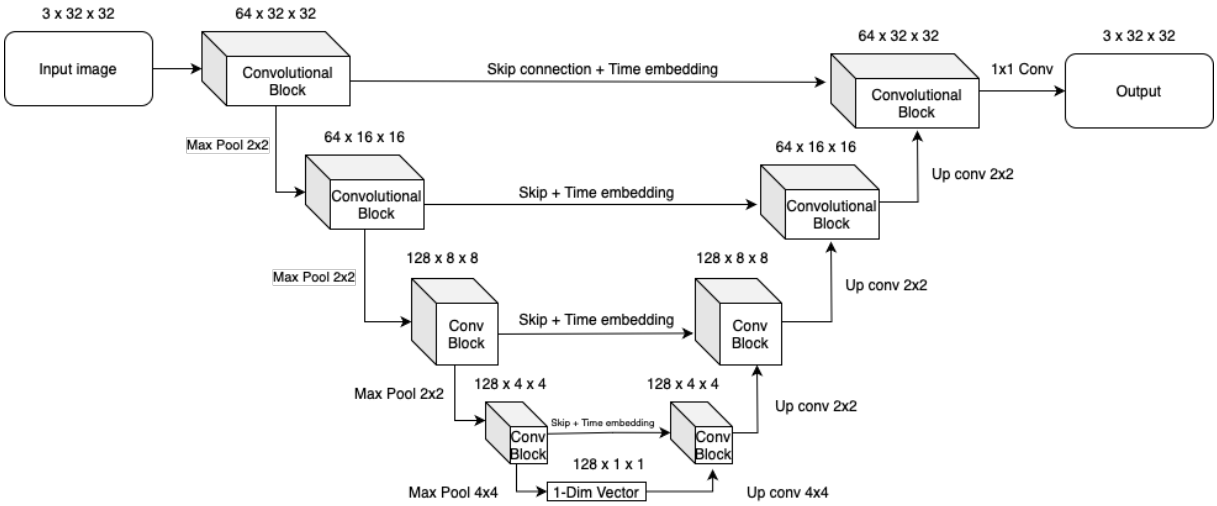


Figure 9: U-NET architecture employed for CIFAR10

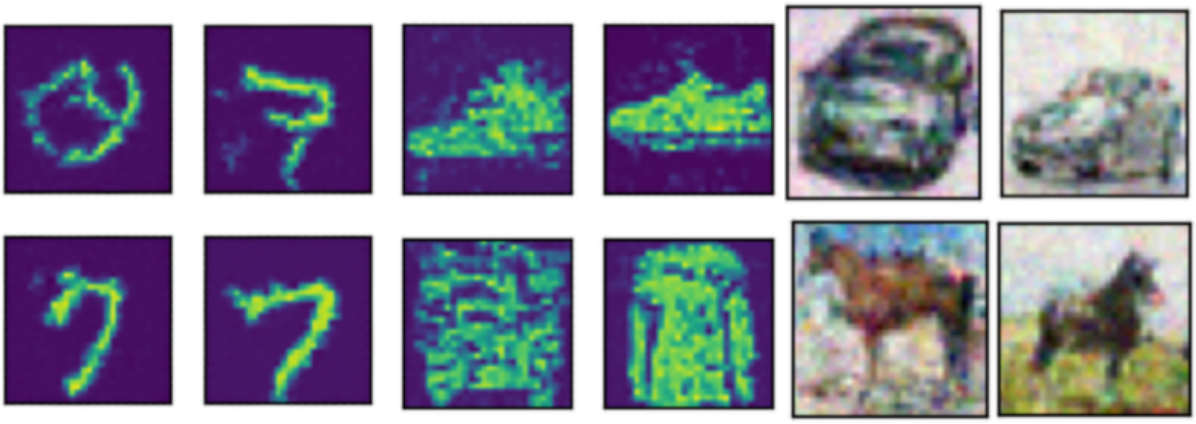
By convolutional block we mean the repeated application of two 3x3 convolutions (padding equal to 1, so that we don't change the dimensions), each followed by a rectified linear unit (ReLU). Also, the embedding of time has been performed by single layer networks.

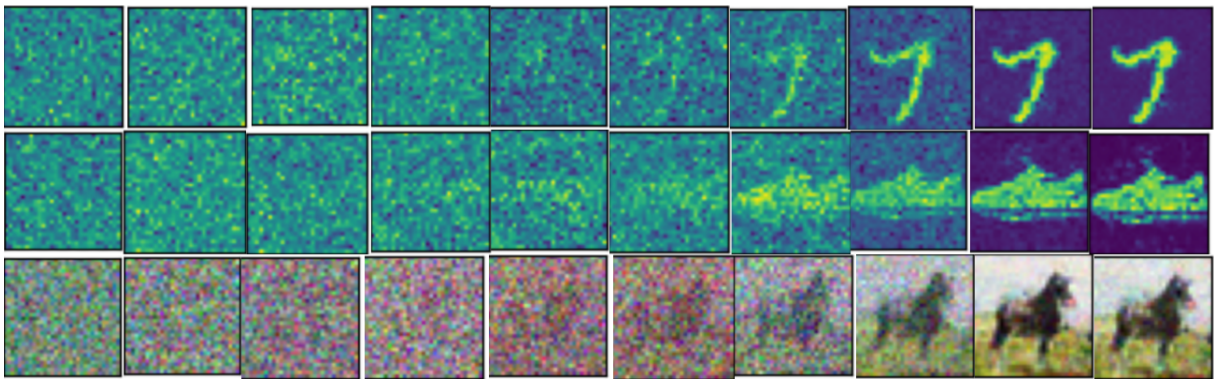
The architecture employed to train the denoisers for the MNIST datasets, instead, is easier. Everything is kept the same except for the number of application of the Max Pool operations: while for CIFAR we use 3 Max Pool 2x2 and 1 Max Pool 4x4, for the others we just use 2 Max Pool 2x2 and 1 Max Pool 7x7 to get to the one dimensional vector quickly.

As for the training, we decided to implement an ADAM optimizer with a fixed learning rate of 10^{-4}. We trained the MNIST datasets and CIFAR for respectively 500 and

8000 epochs, and we used batches of 128 for all of them. All of the images were used for the training and no data augmentation techniques have been employed. Recall that to verify the theoretical values of t_C and t_s we are assumed to use the true score, so we are going to heavily overparametrize our network hoping to get as close as possible to its prediction.

We now show some of the generated images.





Notice that the generated images for CIFAR are still very noised, this means that the model was underparametrized for this task and that the score learned may not be close to the true one.

5.3 Methods

We now proceed to explain in which way we estimated collapse and speciation times.

5.3.1 Speciation

To numerically estimate it, we use the same approach explained in the previous chapter with regards to Gaussian mixtures: we generate two independent trajectories at each time t of the backward process and we estimate the probability $\phi(t)$ that, starting from the same initial conditions, they end their dynamics in the same class. We will then plot the results and see if their behaviour is captured by the theoretical predictions given in Table 1 (note that for the CIFAR dataset, Λ is computed only with regards to the first channel).

Table 1: Theoretical predictions

Dataset	n	d	Λ	t_s
MNIST	12188	28x28	12.14	1.25
FashionMNIST	12000	28x28	31.78	1.73
CIFAR10	10000	3x32x32	16.59	1.40

To check whether or not the two clones end up in the same class, we have to classify the generated samples. In order to do so, we also trained three different neural networks (one for each dataset) to solve that task. The reason why we chose not to use classifiers that already exist, is because a few times it happened that they recognized classes that we removed from the dataset, and this caused different problems in the estimation of $\phi(t)$. These classifiers have a very simple structure, but manage to achieve more than 95% accuracy on CIFAR10 and around 99% on the MNIST datasets (accuracy is obviously computed on the testsets). The architecture used for CIFAR is the following

- Conv2d (3, 6, kernel = (5, 5))
- MaxPool2d (kernel = (2, 2), striding = (2,2))
- Conv2d (6, 16, kernel = (5, 5))
- Linear (16 * 5 * 5, 120)
- Linear (120, 84)
- Linear (84, 2)

The only changes for the MNIST dataset are given by the consequences of the fact that we start from only one channel.

5.3.2 Collapse

As said before, in order to be able to observe the collapse, we have to use a small number of training data (so that the dynamic is ensured to collapse on one of the training points): meaning that we are going to re-train our U-nets using smaller datasets (the main reasons were lack of computational power and time). For MNIST we choose $n = 200$, for FashionMNIST $n = 1000$ and for CIFAR10 we used the whole dataset. To find the collapse time we are going to use a combination of two different approaches, and see if they agree on the estimation.

The first strategy is to use again the cloning procedure, but instead of calculating the ususal $\phi(t)$ we compute $\phi_c(t)$, defined as the probability that the two clones collapse on the same training point. In order to check if the clones collapse in this way, it is enough to make sure that they both have the same training point as nearest neighbour (with respect to Euclidean distance). By plotting $\phi_c(t)$ we may be able to see the interval in which the function goes from zero to one, and thus we can already try to guess t_c.

To adjust our estimation, we use a second strategy that consists of keeping track, for each trajectory, of its closest training point at every t. We then estimate t_c as the last time during the backward process at which the closest neighbour changes (from that point on, the trajectory keeps going closer and closer to it). We will then put together the results of these two different approaches and see if they are equivalent.

5.4 Results and considerations

First, let's recall that the dynamic of the forward process in a DDPM is described by

$$\mathbf{x}_{t'} = \sqrt{\bar{\alpha}_{t'}}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_{t'}}\epsilon,$$

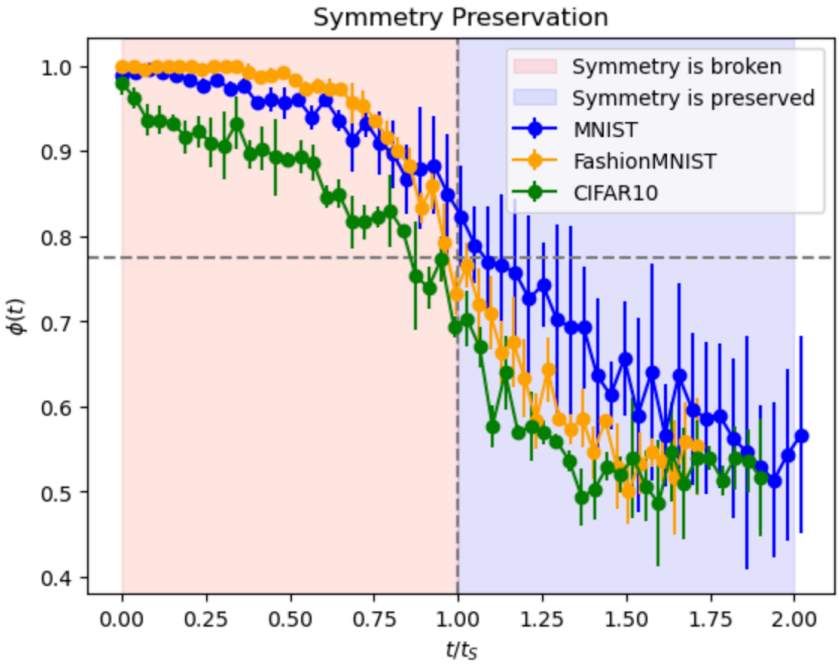
that corresponds to the discretized version of the process 9. We can make them equivalent by reparametrizing the step t' as

$$t = -\frac{1}{2}\log(\bar{\alpha}_{t'}),$$

meaning that $T = 5.05$. In this way we have a correspondence between experimental data and theoretical predictions. Let's now show the experimental results.

5.4.1 Speciation

Let's show the resulting graph. Notice that we rescaled the time axis for each dataset, in order to make them meet in the middle.



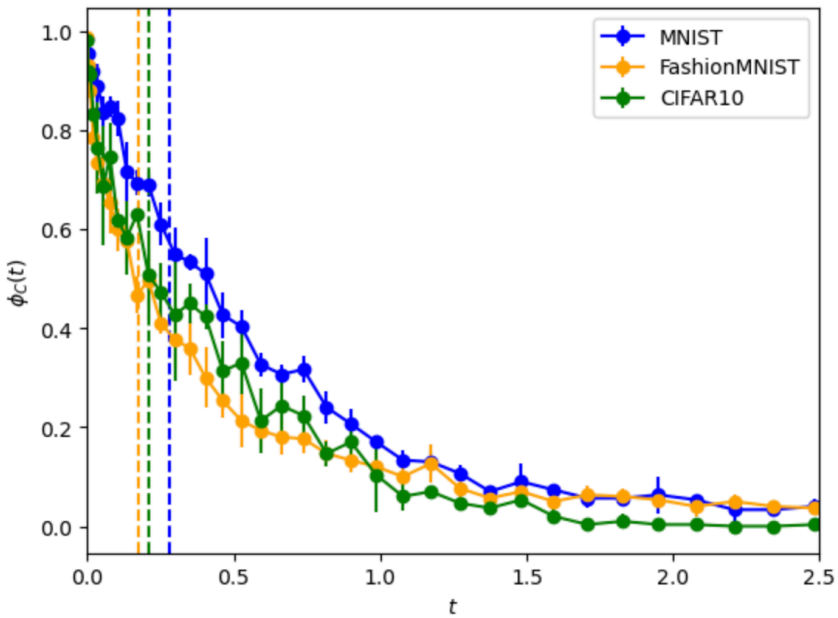
The figure above shows the evolution of $\phi(t)$ as a function of t/t_s. The error bars correspond to the standard error, computed over three different trials (we assume that the low number of trials is the reason why the MNIST line has this much variability). Each trial constitutes of one hundred different couples of trajectories that depart from the same points at different times. The horizontal segmented line corresponds to the true $\phi(t_s)$ for Gaussian mixtures.

As expected, $\phi(t)$ starts at $1/2$ for high times, and increases to 1 as time goes to zero. In this experiment we also analyzed datasets with the same dimension (MNIST and FashionMNIST, whose main difference is Λ), to see if the respective behaviours of the ϕ's depend on other things. What we seem to obtain is that the sigmoid describing FashionMNIST is steeper than the one describing MNIST (it starts off below and ends up above): we can thus speculate that the slope may be positively correlated with the Λ value of the dataset. The CIFAR line, instead, does not respect our expectations, as it lies below the others despite being related to a higher dimensional dataset (it should be steeper of MNIST, at least). We blame this fact on the denoiser, that, as stated before, may be underparametrized (the samples are not that clear, so we expect the intermediate images to be hard to recognize).

In conclusion the theoretical t_s's seem to be in the "middle" of the upward trajectories of the lines, meaning that they can be regarded as acceptable estimations.

5.4.2 Collapse

We now show the results of the collapse experiments.



Datapoints and error lines are defined as before, but notice that the time axis is not

rescaled. The vertical segmented lines correspond to the estimations of t_C given by the ”second approach” explained in the previous section.

Also in this case, the lines for MNIST and FashionMNIST are very similar, and this is because of the fact that the datasets live in spaces with the same dimension number. The behaviour of CIFAR may seem unjustifiable (having more than three times the number of dimensions of the other datasets, it should possess a much higher collapse time), but actually the procedure we implemented has 2 characteristics that allowed the dynamics to shrink the third regime and avoid memorization: the first one is the use of a very approximated score, and the second one is the fact that we trained it on the whole dataset, so we made the α factor bigger than the one we needed.

6 Conclusions

In this survey we managed to cover some of the fundamental aspects of Diffusion-based Generative Models: we defined the basic properties of data, explained how to exploit them in our favor, and finally defined and implemented some algorithms.
Due to time constraints and lack of computational power, we could not completely replicate all of the results stated in Chapter 4. However, the results we obtained are sufficiently aligned with theoretical expectations and we can thus confirm the practical feasibility of the discussed methods.
Our detailed analyses in Chapters 2 and 3 provide a comprehensive foundation in Diffusion Probabilistic Models, ensuring that readers are well-prepared to understand recent advances in the field. The explanations in Chapter 4 aim to offer a high-level understanding of how statistical physics methods are applied to diffusion models.
In conclusion, while our study has provided significant insights into diffusion models, there remains a vast potential for further exploration. Future work could focus on delving into the improvements of the models studied [9] [12], or to repeat the experiments trying datasets with many classes and deeper denoising networks.

References

[1] Giulio Biroli et al. *Dynamical Regimes of Diffusion Models*. 2024. arXiv: 2402.18491 [cs.LG].

[2] Alessio Colombo. *Thesis Experiments Replication*. Version 2.0.4. June 2024. URL: https://github.com/Sland0r/Final_work.

[3] William Feller. "Fluctuation theory of recurrent events". In: *Transactions of the American Mathematical Society* 67.1 (1949), pp. 98–119.

[4] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. 2020. arXiv: 2006.11239 [cs.LG].

[5] Aapo Hyvärinen and Peter Dayan. "Estimation of non-normalized statistical models by score matching." In: *Journal of Machine Learning Research* 6.4 (2005).

[6] Diederik P Kingma and Max Welling. *Auto-Encoding Variational Bayes*. 2022. arXiv: 1312.6114 [stat.ML].

[7] Alex Krizhevsky, Geoffrey Hinton, et al. "Learning multiple layers of features from tiny images". In: (2009).

[8] Yann LeCun et al. "Gradient-based learning applied to document recognition". In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.

[9] Alex Nichol and Prafulla Dhariwal. *Improved Denoising Diffusion Probabilistic Models*. 2021. arXiv: 2102.09672 [cs.LG].

[10] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. *U-Net: Convolutional Networks for Biomedical Image Segmentation*. 2015. arXiv: 1505.04597 [cs.CV].

[11] Yang Song and Stefano Ermon. *Generative Modeling by Estimating Gradients of the Data Distribution*. 2020. arXiv: 1907.05600 [cs.LG].

[12] Yang Song et al. *Score-Based Generative Modeling through Stochastic Differential Equations*. 2021. arXiv: 2011.13456 [cs.LG].

[13] Yang Song et al. *Sliced Score Matching: A Scalable Approach to Density and Score Estimation*. 2019. arXiv: 1905.07088 [cs.LG].

[14] Pascal Vincent. “A Connection Between Score Matching and Denoising Autoencoders”. In: *Neural Computation* 23.7 (2011), pp. 1661–1674. DOI: 10.1162/NECO_a_00142.

[15] Max Welling and Yee W Teh. “Bayesian learning via stochastic gradient Langevin dynamics”. In: *Proceedings of the 28th international conference on machine learning (ICML-11)*. Citeseer. 2011, pp. 681–688.

[16] Han Xiao, Kashif Rasul, and Roland Vollgraf. *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. 2017. arXiv: 1708.07747 [cs.LG].